Chunking

Limited Context Window

Size, Content & Boundaries
- Semantic
- Non Semantic

1) optimizing Retrieval Accuracy

Large :- mixing of multiple idea
Noise & Jitter

Small :- Focused, Single idea

Chunk Size :- 2000 ✗

500 - 1200

2) Preserving the Context

Too small :-

Too big :- Noise

Attention Mechanism

↳ Attention Dilution

↳ lost in the middle effect.

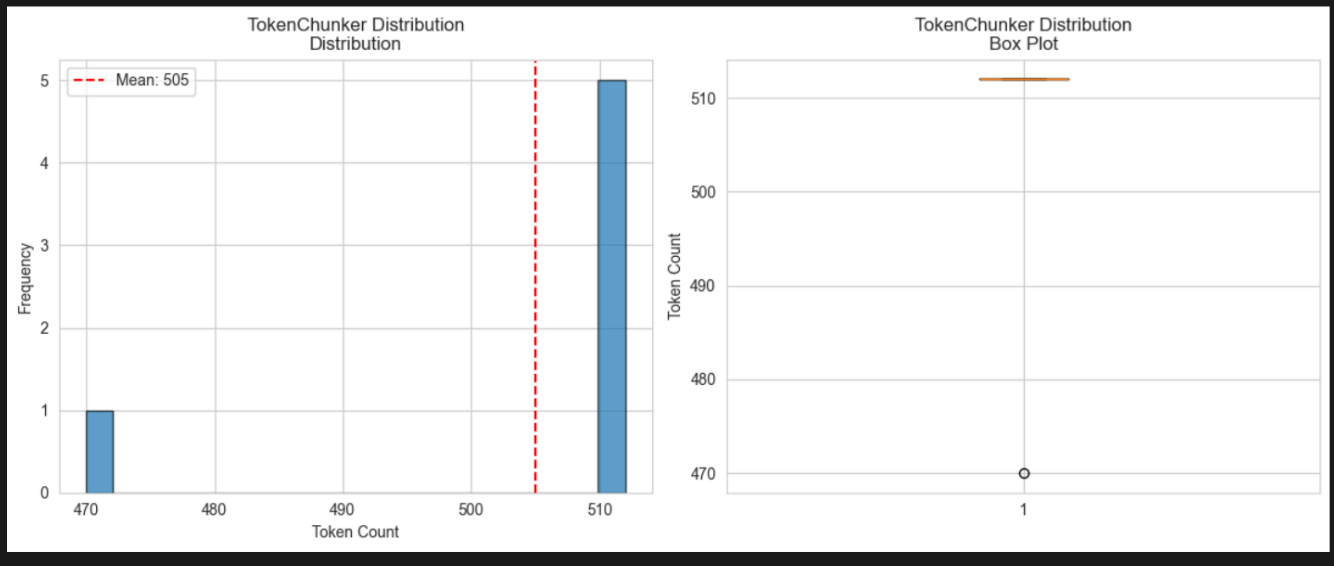
Trouble accessing the information.

* Hallucinating
* Reasoning

Sweet Spot

1) Cost Optimization
2) Making the LLM Context Window
3) Reducing Hallucination
4) Improving

Chunking is part of Context Engineering




```
Statistics:
  Mean: 505.0
  Median: 512.0
  Std Dev: 15.7
  Min: 470
  Max: 512
```

Mean :- 505 (Avg Chunk size) 512

Estimating the storage / memory reqs

Median :- 512

Median > Mean

512 505

Ideal :- 512 close

but few outliers.

Standard Dev : 15.7

Very low (~3% of mean)

Highly consistent

For RAM :- low std. dev

Min - Max (Range)

470 - 512

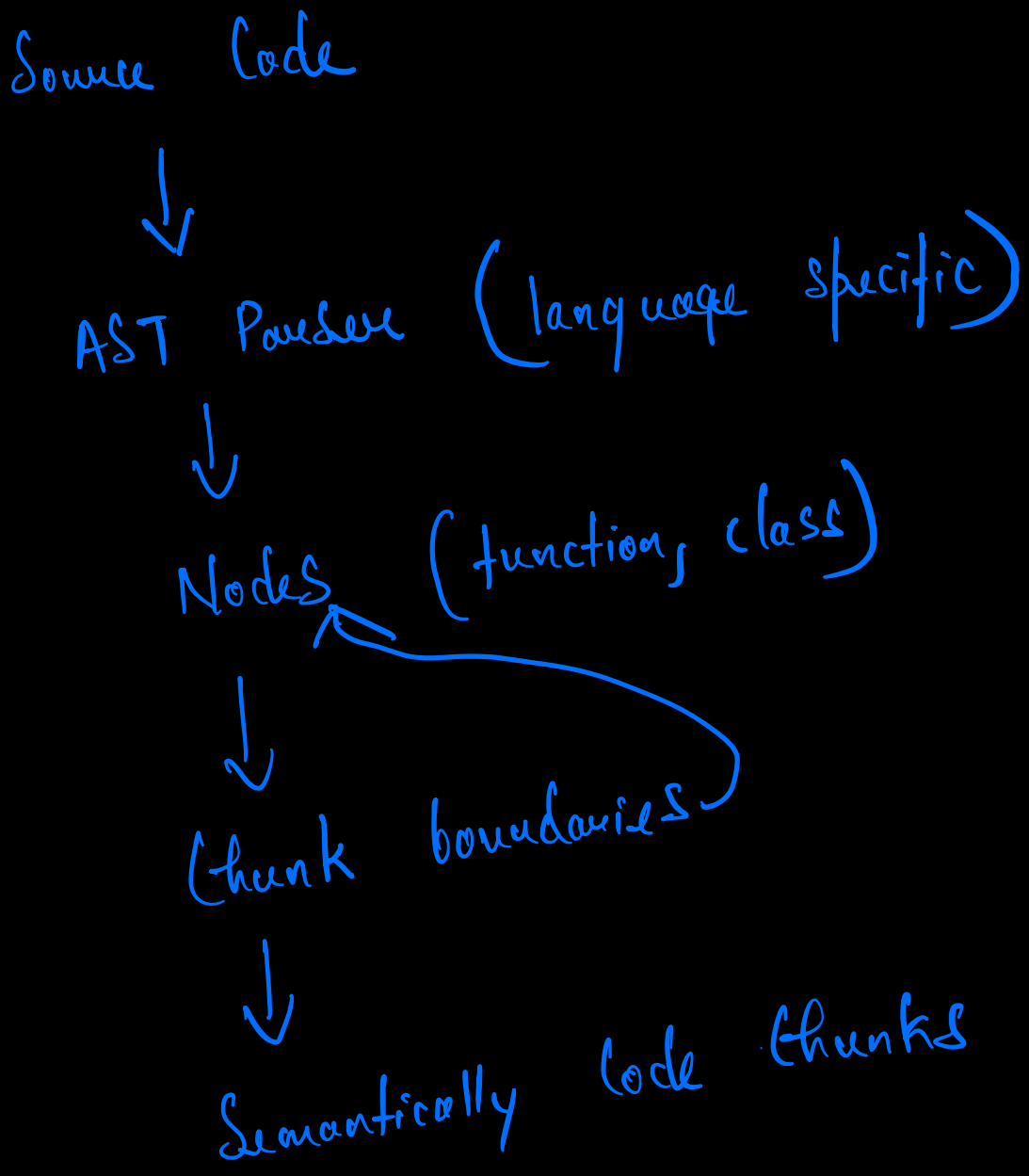
= 42 Token difference

Narrow = Uniform Chunks

Distribution

AST

1) Function
2) Classes
3) Methods
4) Imports
5) Control blocks


Source Code
AST Parser (language specific)
Nodes (function, class)
Chunk boundaries
Semantically Code Chunks

Late Chunking

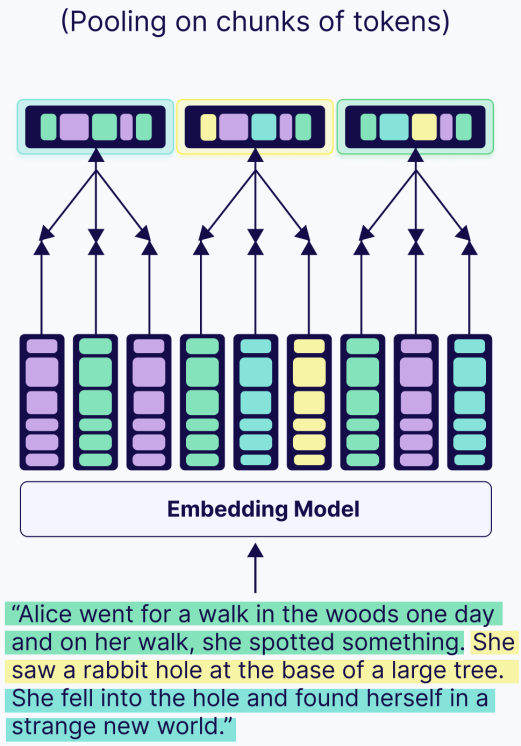
1 major problem :- Context loss

4. **Pool strategically** instead of pooling all tokens into one vector, late chunking pools tokens according to your chunking strategy to get multiple contextually-aware embeddings per document

3. **Preserve context** because the embeddings were created with full document context, each chunk "remembers" its relationship to neighboring chunks.

2. **Chunk the embeddings** (not the text)

1. **Embed the entire document** using a long-context model



- Isolated Chunks
  - Ambiguous
  - Context loss

1. C. Embedding Model

1) Token level embeddings

↳ Full context (Full Picture)

↓

Split

Average the token embeddings

What is Pooling?

1) Mean Pooling

Text

↓

Tokenizer

↓

Encoder → Embedding

↓

POOLING

Compressing

Token level Embedding

↓

Single Fixed Size Vector

(VectorDB) ← 1 chunk = 300 token

↓

300 embeddings

Default :- 1 vector / token

300 -> Single vectors

Mean (Avg) Pooling

$$\text{Chunk\_Embedding} = \frac{1}{N} \sum_{i=0}^{N} e_i$$

Average of all token level embeddings

Sentence Transformers

384, 786, 1536, 3072

Pooled Embedding Dimensions

Average ⇝ Info loss

Neural Chunkers

Semantic Shift → BERT

Doc → Tokens → Token Classification

↙

Identifying Chunk boundaries

512

↳ Positional Encoding Constraint

Agentic LLM Chunker

Semantic Unit → Proposition

More meaning than fixed size chunks

Proposition → An atomic → one idea
self contained factual one
a logical statement.

Linguistics (NLU)

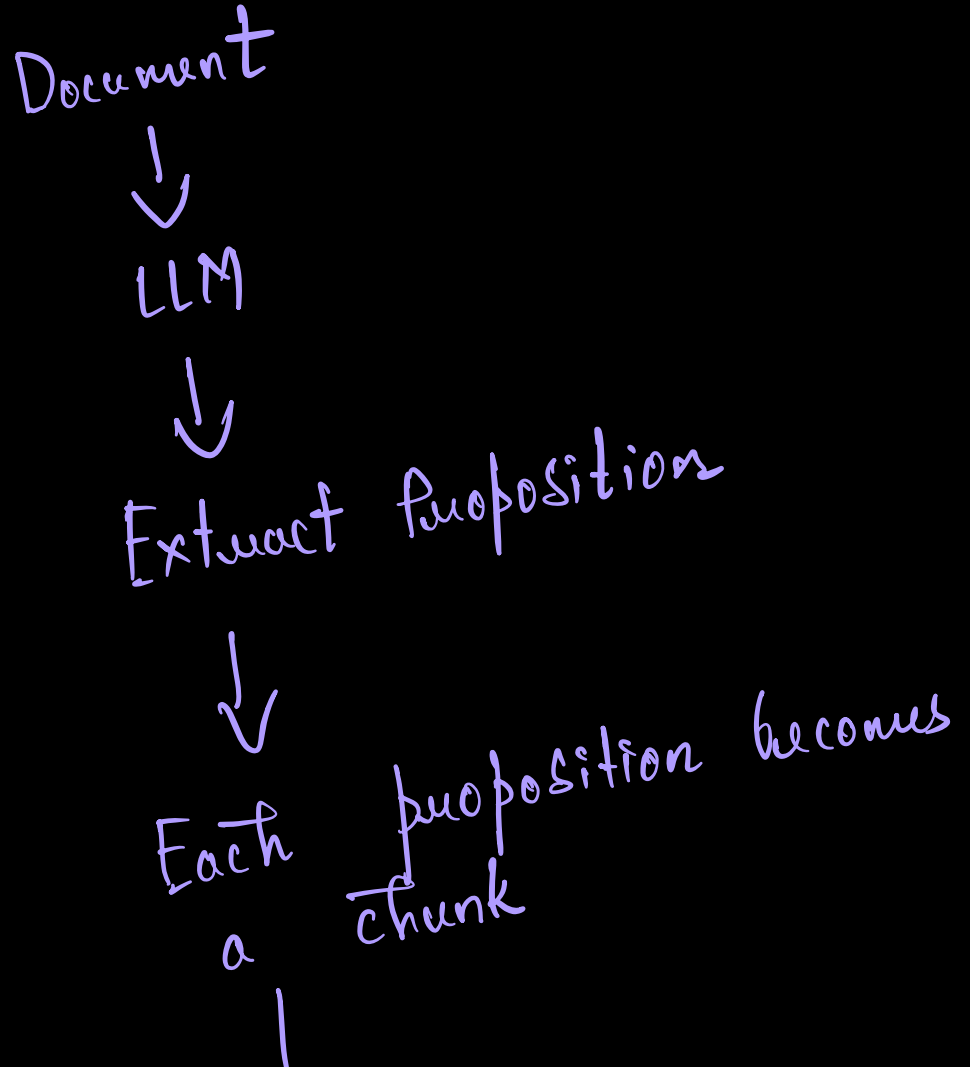
Example :-

Sent 1 Paul is going to london. Paul loves football. → Sent 2

Proposition :- Sent (1) claim.
Sent (2) claim.

Each line is a proposition.

LLM based Proposition Chunking



Save it VectorDB

C_1 P_1

C_2 P_2

C_3 P_3

C_4 P_4

Retrieval Accuracy ↑

less Hallucinations

Better Factual Grounding

{* Hybrid * Reranking}

HYDE

Hypothetical Document Embeddings

{Zero shot, Dense Retrieval}

1) Generate a hypothetical document using a LLM

2) Encode this document

↓

Contrastive learning

ML

Differentiate b/w similar & dissimilar data points.

learn → Representation Documents

High dimension

(Cluster) Similar Docs → close

(Cluster) Dissimilar Docs → Far away

Efficient Similarity Retrievals Systems.

* MRL *

Hyde

1) Query Input
2) Hypothetical Document Generation
   - Relevance Patterns
   - Errors

3) Document Encoding

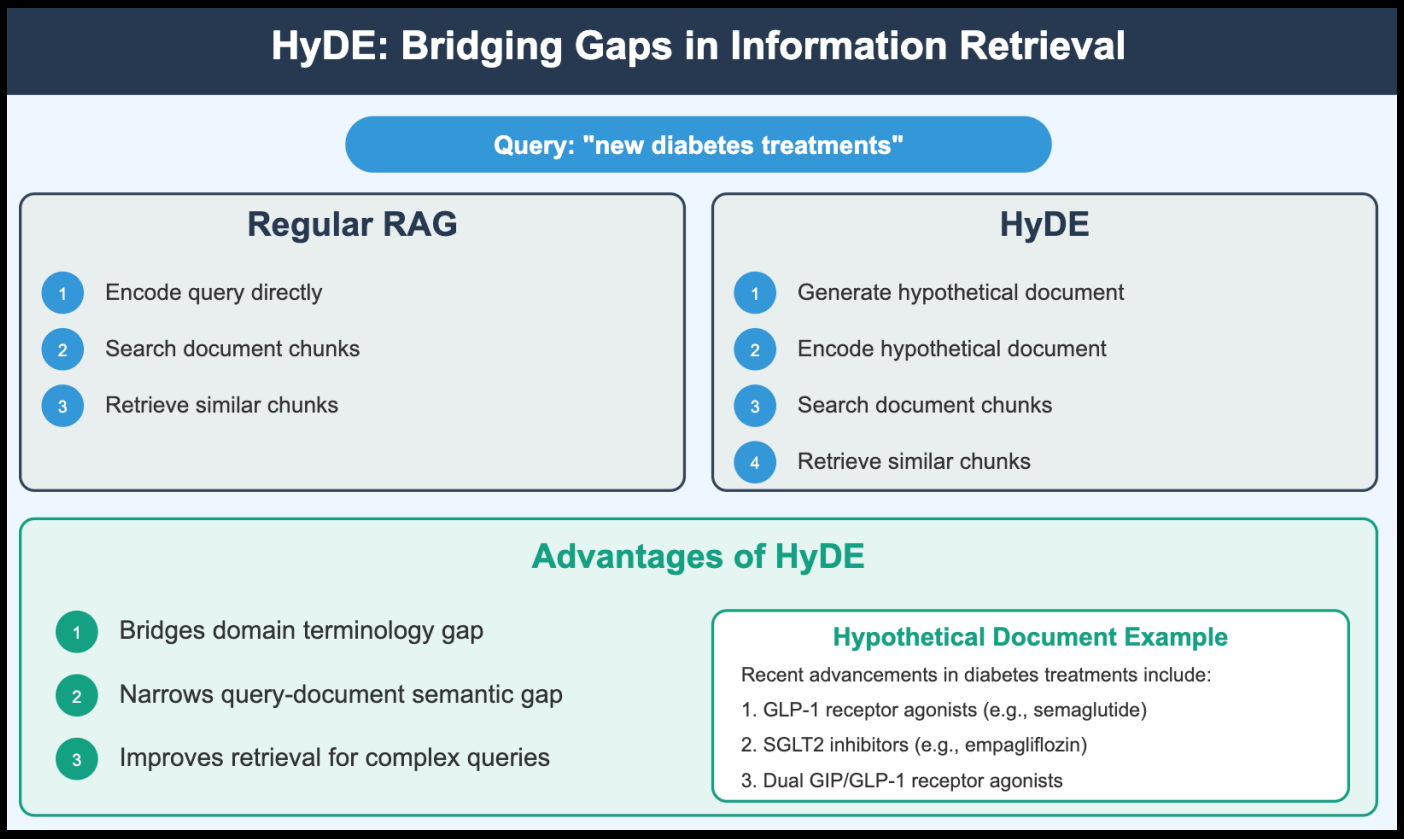
{Contriever}

↙ constrastive Encoder (Unsupervised)

↘ Embeddings

4) Similarity Search → Real Docs

5) Retrieval Results



Dense Vs Sparse

Dense → low dim (non-zero)

Sparse → very high dim (mostly zero)

D (128 - 4096)

S (10000 - 50000)

D → Word2Vec, Glove, BERT, Open AI Embeddings, CNN, ViT

(D) Advantages :-

1) Strong Semantic Understanding
2) Similarity Search.
3) Compact Storage

Disadvantages

1) Hard to Interpret
2) Costly to compute.

Sparse

Algos :- BOW, TFIDF, OHE, BM25

Why many zeros ?

Vocabulary Size

1) Cheap Computation {Fast Embed}

Exact Matching

Sparse + Dense ⇒ Hybrid Search